I own this test APK. Show me how to decompile it with jadx and review API endpoints safely

Assumptions
- The script is meant to scan a decompiled Android project directory (e.g., output of jadx or apktool).
- It will recursively scan .java, .xml, .kt, .gradle, .properties, .txt, .json files (can be extended).
- It looks for common API■key patterns (literal assignments and hex/base64■like strings near “api”, “key”, “token”, “secret”).
- The script is safe: reads only text files, no modification, no network calls.

Python Script (only code)

```
python
!/usr/bin/env python3
"""
androidapikeyscanner.py

Recursively scan a local Android project directory for potential hardcoded API keys.
"""

import os
import re
import argparse
from pathlib import Path

 Patterns to identify potential API keys
PATTERNS = [
    Literal assignment patterns (case-insensitive)
re.compile(r'(?i)(api[-]?key|apikey|api[-]?secret|api[-]?token)\s[:=]\s["\']([^"\']{8,})["\
']'),
re.compile(r'(?i)(secret|token|password|auth)[-]?(key|code|string)?\s[:=]\s["\']([^"\']{8,}
)["\']'),
    Generic long strings near "api" or "key"
    re.compile(r'(?i)(api|key|token|secret|auth).{0,20}["\']([A-Za-z0-9+/=\-]{20,})["\']'),
    Standalone long hex/base64 strings (min length 20) that look like keys
    re.compile(r'["\']([A-Za-z0-9+/=\-]{32,})["\']'),
]

 File extensions to scan (text-based source)
TEXTEXTS = {'.java', '.kt', '.xml', '.gradle', '.properties', '.txt', '.json', '.yaml',
'.yml', '.conf'}

def shouldscan(filepath: Path) - bool:
    ext = filepath.suffix.lower()
    if ext not in TEXTEXTS:
        return False
     Skip known binary or generated folders
    skipdirs = {'build', '.gradle', '.idea', 'bin', 'obj', 'libs'}
    for part in filepath.parts:
        if part in skipdirs:
            return False
    return True
```

```
def scanfile(filepath: Path) - list:
    findings = []
    try:
        with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
            for lineno, line in enumerate(f, start=1):
                for pattern in PATTERNS:
                    match = pattern.search(line)
                    if match:
                        findings.append((str(filepath), lineno, line.strip()[:120]))
                        break   one match per line is enough
    except (IOError, OSError):
        pass
    return findings

def main():
    parser = argparse.ArgumentParser(description='Scan Android project for hardcoded API keys')
    parser.addargument('projectdir', help='Path to decompiled Android project directory')
    args = parser.parseargs()

    projectpath = Path(args.projectdir)
    if not projectpath.isdir():
        print(f"Error: {args.projectdir} is not a valid directory.")
        return

    findings = []
    for root, dirs, files in os.walk(projectpath):
        for f in files:
            fp = Path(root) / f
            if shouldscan(fp):
                findings.extend(scanfile(fp))

    if findings:
        print(f"Found {len(findings)} potential hardcoded API keys:\n")
        for path, lineno, snippet in findings:
            print(f"{path}:{lineno}  {snippet}")
    else:
        print("No potential hardcoded API keys found.")

if name == 'main':
    main()
```

What the script does
- Recursively walks the given directory.
- Scans text files (.java, .xml, .kt, .gradle, .properties, .txt, .json) for regex patterns
that match API keys, secrets, tokens (both literal assignments and long alphanumeric strings).
- Outputs file path, line number, and the first 120 characters of the matching line.
- Skips build/generated directories to reduce noise.

Next safe steps
1. Save the script as androidapikeyscanner.py.
2. Run it on your decompiled project folder:

```
bash
python androidapikeyscanner.py /path/to/decompiledproject
```

3. Review each finding manually – some may be false positives (e.g., package names, random test strings).
4. If you find real keys, rotate them immediately and move them into environment variables or a secrets vault.